

1) Explain Programming and Python in detail?

Programming is the process of writing instructions that tell a computer what to do. These instructions help the computer solve problems or perform tasks like calculations, data storage/decision-making.

Example:

A program to add two numbers

```
a = 10
```

```
b = 20
```

```
print(a+b)
```

What is Python?

Python is a high-level programming language that is easy to learn and understand. It was developed by **Guido Van Rossum** and is widely used because of its simple syntax and readability.

Characteristics of Python

- ⇒ Easy to read and write
- ⇒ Uses simple English-like syntax
- ⇒ Large collection of libraries
- ⇒ Free and open-source
- ⇒ Supports multiple programming styles

Applications of Python

- ⇒ Web development
- ⇒ Data analysis
- ⇒ Automation
- ⇒ Scientific research

Example:

```
print("Hello, World")
```

Types of Comments in Python

Comments are statements in a program that are not executed by Python. They are used to explain the code and improve readability.

1. Single-line Comments

- ⇒ Used to explain a single line of code
- ⇒ Begins with the `#` symbol.
- ⇒ Python ignores this line during execution

Syntax:

```
# This is a single-line comment
```

Example:

```
# Assign value to variable  
x = 10
```

2. Multi-line Comments

- ⇒ Used to explain multiple lines at once
- ⇒ Written using triple quotes (`'''` or `"""`)
- ⇒ Commonly used for documentation.

Syntax

```
"""  
This is a  
multi-line comment  
"""
```

Example

```
"""  
This program calculates sum  
of two numbers  
"""  
a = 5  
b = 6  
print(a+b)
```


Importance of Comments

- ⇒ Makes code easy to understand
- ⇒ Helps in debugging
- ⇒ Useful for documentation

Importance of Python in Modern Software Development

- ⇒ It is easy to learn and use
- ⇒ Simple syntax reduces coding time
- ⇒ Supports multiple programming styles
- ⇒ Has many libraries and frameworks
- ⇒ Used in AI, data science, web development, and automation
- ⇒ Works on different operating systems.

Data Types and Operators in Python

Python provides different built-in data types to store various kinds of data and operators to perform operations on them.

Built-in Data Types in Python

1. Numeric Data Types

Used to store numbers such as integers and decimal values.

```
a = 10    # integer
b = 3.5    # float
```

2. Sequence Data Types - Tuple - ordered collection; immutable

Used to store a collection of values

```
name = "Deepu"    # string
marks = [80, 85, 90] # list - ordered collection; Mutable
```

3. Set Data Type

stores unique elements

```
nums = {1, 2, 3}
```

List	-	[]
Tuple	-	()
Set	-	{ }

4. Mapping Data Type

Stores data in key-value pairs

```
student = { "name": "Deepu", "age": 20 }
```

5. Boolean Data Type

stores True or False values

result = True

int $\Rightarrow a=10, b=-12, c=12$ 2 3 4 5 6 7

Float = $x = 1.0$; $y = 12.3$; $z = 13.4$

Complex = Alphabet with number ; $A = 2 + (5j)$
 $\begin{matrix} \text{real} & \text{imaginary} \\ \text{number} & \end{matrix}$

String - Sequence of characters represented in quotation marks. In python we can use single, double/triple quotes to define a string.

"hello python".

Type Identification using `type()`

The `type()` function in python is used to identify the data type of a variable. It tells what kind of value is stored in the variable.

type (variable_name)

$$a = 10$$

```
print(type(a)) # int
```

$$b = 3.5$$

```
print(type(b)) # Float
```

name = "Python"

```
print (type(name)) # str
```

nums = [1, 2, 3]

```
Print (type (nums)) # list
```

flag = True

```
print(type(flag)) # bool
```


Operators in Python ($x \text{ [Operator] } y$) → operands

= Used to perform operations on variables & values
= Python operators are special symbols used to perform specific operations on one or more operands.

Unary Operators - Python operators that require one operand to perform a specific operation known as Unary operators.

Binary Operators - Python operators that require two operands to perform a specific operation.

Operands - Variables, values or expressions that are used with operator to perform a specific operation.

Types of Python Operators

Python operators are categorized in the following categories.

1. Arithmetic Operators

Used for mathematical calculations

Operator	Meaning	Example
+	Addition	$10 + 5 = 15$
-	Subtraction	$10 - 5 = 5$
*	Multiplication	$10 * 5 = 50$
/	Division	$10 / 5 = 2.0$
%	Modulus (remainder)	$10 \% 3 = 1$
//	Floor Division	$10 // 3 = 3$
**	Exponent	$2 ** 3 = 8$

Real-world use:

Calculating total marks, salary, discounts, average score, etc.

2. Assignment Operators

Used to assign values to variables

Operator

Example

Meaning

=

$x = 10$

Assign value

+=

$x += 5$

$x = x + 5$

- =

$x -= 5$

$x = x - 5$

* =

$x *= 2$

$x = x * 2$

Real-world use:

Updating Bank Balance, Increasing counters, modifying scores.

3. Comparison Operators

Used to compare two values. Result is True / False

Operator

Meaning

==

Equal

!=

Not equal

>

Greater than

<

Less than

>=

Greater than or equal

<=

Less than or equal

Real-world use:

Checking pass / fail marks, age eligibility, login validation.

4. Logical Operators

Used to combine conditions

Operator

Meaning

and

True if both conditions are true

or

True if any one condition is true

not

Reverses the result.

Real-world use:

Checking multiple conditions like age and qualification

Membership Operators

Used to check whether a value exists in a sequence

Operator

Meaning

in

Value exists

not in

Value does not exist

Example - 'a' in apple # True

Real-world use:

Checking username in database, item in list.

Identity Operators - used to compare memory locations

is

Same object

is not

Different object

Real-world use:

Checking object identity in programs

Python Input and Output Operations

Input Operation in Python

input() Function

⇒ used to take input from the user

⇒ Default data type is string (str)

```
name = input("Enter your name:")
```

Type Conversion While Taking Input

Since input is always string, type conversion is required.

```
age = int(input("Enter age: "))
```

```
salary = float(input("Enter salary: "))
```

Taking Multiple Inputs

Multiple inputs can be taken in one line using `split()`

```
a, b = input("Enter two numbers: ").split()
```

```
a = int(a)
```

```
b = int(b)
```

Output Operation in Python

`print()` Function

Used to display output on the screen.

```
print("Hello Python")
```

Separator (sep)

Used to separate multiple values

```
print(10, 20, 30, sep = " , ")
```

Output

10, 20, 30.

Format Specifiers / Formatted Output

Using `format()`

```
print("My age is {}".format(age))
```

Using f-string

```
print(f"My salary is {salary}")
```

Real-world use:

Displaying reports, student details, invoices, bills.

Control Statements and Decision-Making Statements:

Meaning of Control Statements

They control the flow of execution of a program.

They decide which statement to execute and when.

Importance

- ⇒ Enables decision making
- ⇒ Makes programs logical and dynamic
- ⇒ Avoids unnecessary execution.

Types of Control Statements

1. Decision-making statements
2. Looping statements
3. Jump statements

Decision-Making Statements

1. If - statement

Executes a block of code only if the condition is true.

Syntax

```
if condition:  
    statement
```

Example

```
if marks >= 35:  
    print("Pass").
```

2. If - else statement

Executes one block if condition is true, otherwise executes another block.

Syntax

```
if condition:  
    statement 1
```

```
else  
    statement 2
```

Example

```
if age >= 18  
    print("Eligible to vote")  
else  
    print("Not eligible")
```

3. If - elif - else statement
Used to check multiple conditions

Syntax

```
if condition 1:  
    statement 1
```

```
elif condition 2:  
    statement 2
```

```
else statement 3
```

Example

```
if marks >= 75:  
    print("Distinction")  
elif marks >= 60:  
    print("First class")  
elif marks >= 35:  
    print("Pass")  
else:  
    print("Fail")
```

Execution Flow

- ⇒ Conditions are checked top to bottom
- ⇒ First true condition is executed
- ⇒ Remaining conditions are skipped.

Example: (If - else execution Flow)

1. Condition $age \geq 18$ is checked
2. Condition is False
3. else block executes
4. Program continues after if - else.

Python executes only the block whose condition is true.

Indentation plays a crucial role in controlling flow

Write an Essay on Python programming fundamentals.

1. Role of Programming in Problem Solving

⇒ Programming helps in solving real-world problems using logical steps.

⇒ It breaks complex problems into smaller, manageable tasks

⇒ Enables automation of repetitive work

⇒ Helps in accurate calculations and data processing.

⇒ Improves efficiency and saves time.

2. Python Syntax Simplicity and Readability

⇒ Python has simple and English-like syntax

⇒ Easy to learn for beginners

⇒ Does not use braces {} or semicolons ;

⇒ Uses indentation to define code blocks

⇒ Programs are easy to read, write & maintain

3. Use of Comments for Code Documentation

⇒ Comments explain the purpose of code statements.

⇒ Improve readability and understanding of programs.

⇒ Helpful for debugging and future modifications

⇒ Single-line comments use #

⇒ Multi-line comments use triple quotes (''' or """).

⇒ Comments are not executed by the interpreter.

4. Data Types, Operators and Input/Output Operations

⇒ Python supports built-in data type like

int, float, str and bool.

⇒ Operators perform arithmetic, comparison, logical and assignment operations.

⇒ `input()` function is used to take input from the user.

⇒ Default data type of `input()` is "string"

⇒ `print()` function is used to display output

⇒ Supports formatted output for better presentation

5. Control Flow Using Decision-Making Statements

⇒ Control Flow decides the order of execution in a program.

⇒ Decision-making statements help in making logical choices.

⇒ `if` statement executes code when condition is true.

⇒ `if-else` provides two-way decision making

⇒ `if-elif-else` handles multiple conditions

⇒ Makes programs flexible and dynamic.

Conclusion

⇒ Python fundamentals form the base for advanced programming

⇒ Simple syntax and powerful features make python popular

⇒ Widely used in education, science, data analysis and software development.

1) Movie Ticket Pricing

A movie theatre charges;

₹150 for children (age < 13)

₹250 for adults (age 13-59)

₹200 for seniors (age ≥ 60)

If the person is watching a 3D movie, add ₹50 extra

Write a program that takes age and is 3D (1 or 0) and prints the final ticket price.

```
age = int(input("Enter age: "))
```

```
is3D = int(input("Is it a 3D movie? (1 for Yes, 0 for No): "))
```

```
if age < 13:
```

```
    price = 150
```

```
elif age <= 59:
```

```
    price = 250
```

```
else
```

```
    price = 200
```

```
if is3D == 1
```

```
    price += 50
```

```
print("Final Ticket price: ₹", price)
```

2) College Attendance Rule: A student is allowed to write the exam if;

attendance ≥ 75

OR

attendance ≥ 60 and has medical certificate (1 = yes, 0 = no)

Take attendance percentage and medical certificate as

input and print "Allowed" or "Not Allowed".

```
attendance = int(input("Enter attendance percentage: "))
```

```
medical = int(input("Medical certificate (1 for Yes, 0 for No): "))
```

```
if attendance >= 75 or (attendance >= 60 and medical == 1):
```

```
    print("Allowed")
```

```
else:
```

```
    print("Not Allowed")
```

3) E-commerce Discount

A shopping site gives

20% discount if bill ≥ 5000

0% discount if bill between 2000 and 4999

no discount if bill < 2000

But if the customer is a prime number they get extra

5% discount

input: bill amount ; Is Prime (1 or 0)

Print final amount to be paid

bill = float(input("Enter bill amount: "))

if bill >= 5000:

discount = 0.20

elif bill >= 2000:

discount = 0.10

else discount = 0.0

isPrime = int(input("Is the customer a Prime member? (1 for Yes, 0 for No): "))

if isPrime == 1:

discount += 0.05

final_amount = bill - (bill * discount)

print("Final amount to be paid: ₹", final_amount)

Smartphone Battery Warning

A phone shows:

"Low Battery" if battery ≤ 20

"Normal" if battery between 21-80

"Full" if battery > 80

But if phone is charging, it should show "Charging" instead of any message.

input: battery percentage, is charging (1 or 0)

battery = int(input("Enter battery percentage: "))

isCharging = int(input("Is the device charging? (1 for Yes, 0 for No): "))

if isCharging == 1:

Print("charging")

if battery ≤ 20:

Print("Low Battery")

Driving license Check

A person can get a driving license if:

age ≥ 18

AND

passed driving test (1=yes)

But if age ≥ 60 , driving test is not required

Input age, test passed

Print "Eligible" or "Not Eligible".

```
age = int(input("Enter age: "))
```

```
testPassed = int(input("Passed driving test? (1 for Yes, 0 for No): "))
```

```
if age  $\geq 60$ :
```

```
    print("Eligible")
```

```
elif age  $\geq 18$  and testPassed == 1:
```

```
    print("Eligible")
```

```
else:
```

```
    print("Not Eligible")
```

Online Food Delivery

A restaurant gives free delivery if:

order amount ≥ 500

OR

user is a gold member

But if the distance is more than 10km, delivery is never free.

Input: amount, is Gold (1 or 0), distance

```
amount = float(input("Enter order amount: "))
```

```
isGold = int(input("Is Gold member? (1 for Yes, 0 for No): "))
```

```
distance = float(input("Enter delivery distance (km): "))
```

```
if distance  $> 10$ :
```

```
    print("Delivery charged")
```

```
elif amount  $\geq 500$  or isGold == 1:
```

```
    print("Free Delivery")
```

```
else:
```

```
    print("Delivery charged")
```

Bank loan Approval

A bank approves a loan if:

salary $\geq 30,000$ AND credit score ≥ 700

OR

salary $\geq 50,000$ (credit score ignored)

Input: salary, credit score

Print "loan Approved" or "loan Rejected".

salary = int(input("Enter salary: "))

creditscore = int(input("Enter credit score: "))

if salary ≥ 50000 or (salary ≥ 30000 and creditscore ≥ 700):

else:

print("loan Rejected")

Electricity Bill

Units consumed

First 100 units $\Rightarrow \text{₹} 2/\text{unit}$

Next 100 units $\Rightarrow \text{₹} 3/\text{unit}$

Above 200 units $\Rightarrow \text{₹} 5/\text{unit}$

Note: No loops

Print final bill amount

units = int(input("Enter units consumed: "))

if units ≤ 100 :

bill = units * 2

elif units ≤ 200 :

bill = (100 * 2) + (units - 100) * 3

else:

bill = (100 * 2) + (100 * 3) + (units - 200) * 5

print("Electricity Bill Amount: ₹", bill)

Student Scholarship

A student gets a scholarship if:

marks ≥ 85

AND

family income < 500000

But if the student is a single parent child, income condition is ignored.

Input: marks, income, singleParent (1 or 0)


```
marks = int(input("Enter marks: "))
income = int(input("Enter family income: "))
singleParent = int(input("Single parent child? (1 for Yes, 0 for No): "))
```

```
if marks >= 85 and (income < 500000 or singleParent == 1):
    print("Scholarship Granted")
else:
    print("Scholarship Not Granted")
```

Online Exam Result

A student gets a scholarship

A student passes if:

theory ≥ 40 AND practical ≥ 40

But if total (theory + practical) ≥ 100 , pass even if one is less than 40.

Input: theory, practical

```
theory = int(input("Enter theory marks: "))
```

```
practical = int(input("Enter practical marks: "))
```

```
if (theory >= 40 and practical >= 40) or theory + practical >= 100:
    print("Pass")
```

```
else:
    print("Fail")
```

Hotel Room Pricing

A hotel charges:

₹ 3000 per day for normal days

₹ 4000 per day on weekends

if customer stays more than 3 days, give 15% discount

Input: isWeekend (1 or 0), daysStayed

Print final bill.

```
isWeekend = int(input("Is it a weekend stay? (1 for Yes, 0 for No): "))
```

```
daysStayed = int(input("Enter number of days stayed: "))
```

```
if isWeekend == 1:
    rate = 4000
```

```

else:
    rate = 3000
    bill = rate * daysStayed
    if daysStayed > 3:
        bill = bill * 0.85
    print("Total bill amount: ₹", bill)

```

2 Gaming Level Unlock

A game unlocks next level if:

score ≥ 100

OR

player has a premium pass

But if player used cheating, access is denied

Input: score, isPremium, usedCheat

score = int(input("Enter score: "))

isPremium = int(input("Is premium player? (1 for Yes, 0 for No): "))

if usedCheat == 1:

print("Access Denied")

elif score ≥ 100 or isPremium == 1:

print("Next level unlocked")

else:
 print("Level locked")

3 Mobile Data Usage

A network gives unlimited data if:

daily usage ≤ 2 GB:

OR

user has unlimited plan

But if roaming is on, unlimited plan does not work.

Input: data Used, hasUnlimited plan, isRoaming

dataUsed = float(input("Enter daily data Usage (GB): "))

hasUnlimitedPlan = int(input("Has Unlimited plan? (1 for Yes, 0 for No): "))

isRoaming = int(input("Is roaming on? (1 for Yes, 0 For NO): "))


```

if isRoaming == 1:
    if dataUsed <= 2:
        print("Unlimited Data")
    else:
        print("limited Data")
else:
    if dataUsed <= 2 or hasUnlimitedPlan == 1:
        print("Unlimited Data")
    else:
        print("limited Data")

```

Office Entry System

An employee can enter the office if:
ID Card is valid

AND

(fingerprint matches OR face scan matches)

But if it is a holiday, entry is denied for everyone.

Input: idValid, fingerprint, facedscan, istholiday

```

idValid = int(input("ID card valid? (1 for Yes, 0 for No): "))
fingerprint = int(input("Fingerprint match? (1 for Yes, 0 for No): "))
facedscan = int(input("Face scan match? (1 for Yes, 0 for No): "))
istholiday = int(input("Is today a holiday? (1 for Yes, 0 for No): "))

```

```

if isHoliday == 1:
    print("Entry Denied")
elif idValid == 1 and (fingerprint == 1 or facedscan == 1):
    print("Entry Allowed")
else:
    print("Entry Denied")

```

Movie Rating Display

A movie app shows rating based on average score:

Average $\geq 8.5 \Rightarrow$ "Excellent"

Average between 6.0 and 8.4 $\Rightarrow$ "Good"

Average $< 6.0 \Rightarrow$ "Average"

but if the movie is marked as editor's choice, always show "Recommended"

Input: averageRating, isEditorsChoice(1 or 0)

Print the message.

```
averageRating = float(input("Enter average rating: "))  
isEditorsChoice = int(input("Is Editor's choice? (1 for Yes,  
0 for No):"))
```

```
if isEditorsChoice == 1:  
    print("Recommended")
```

```
elif averageRating >= 8.5:  
    print("Excellent")
```

```
elif averageRating >= 6.0:  
    print("Good")
```

```
else:  
    print("Average")
```